Magyar Feltalálók Webalkalmazás

Teljes Stack Fejlesztési Projekt – Részletes Beszámoló

Projekt dokumentáció

2026

Buday Benedek - Kovács Kristóf

OXQLH4 - XADNA6

Tartalomjegyzék

1. Bevezetés és a projekt célja
2. Általános architektúra és rendszerterv
3. Backend részletes leírása
4. Frontend részletes leírása
5. Technikai stack részletesen
6. Működési elv és adatfolyam
7. Telepítés és futtatás
8. Tesztelés
9. Biztonság és teljesítmény
10. Fejlesztési megfontolások
11. Összefoglalás és értékelés
12. Képek

Weboldal link:

https://react-crud-app-1.preview.emergentagent.com

1. Bevezetés és a projekt célja

A Magyar Feltalálók Webalkalmazás egy átfogó, teljes stack fejlesztési projekt, amelyet azzal a céllal hoztunk létre, hogy bemutassuk a modern webfejlesztés számos aspektusát egyetlen, összefüggő rendszerben. A projekt központi témája a magyar feltalálók adatainak kezelése, amely tematikailag koherens hátteret biztosít az alkalmazás funkcióinak demonstrálásához. Az alkalmazás valójában egy oktatási és gyakorlati eszköz, amely a JavaScript, a React keretrendszer és a különböző backend technológiák implementációját mutatja be valós, működő kódpéldákon keresztül.

A projekt ötlete abból a felismerésből született, hogy a webfejlesztési technológiák gyors fejlődése miatt egyre nehezebb egyetlen, koherens forrásból megismerni a modern fejlesztési gyakorlatokat. A hagyományos oktatási anyagok gyakran elszigetelt példákon keresztül mutatják be az egyes technológiákat, ami megnehezíti a diákok és a kezdő fejlesztők számára, hogy átlássák, hogyan illeszkednek össze ezek az elemek egy valódi alkalmazásban. Ez a projekt pont ezt a hiányosságot kívánja orvosolni.

1.1 A projekt fő célkitűzései

- Különböző frontend keretrendszerek és könyvtárak összehasonlító bemutatása egyetlen alkalmazáson belül, lehetővé téve a fejlesztők számára, hogy ugyanazt a feladatot különböző megközelítésekkel oldják meg, és közvetlenül összehasonlítsák az eredményeket.
- Backend API fejlesztés a FastAPI keretrendszerrel, amely a Python nyelv egyik legmodernebb és leggyorsabb web keretrendszere, beépített automatikus dokumentáció generálással és típusellenőrzéssel.
- Adatbázis-kezelés MongoDB segítségével, amely NoSQL dokumentum-orientált adatbázisként rugalmas adatstruktúrát biztosít, és kiválóan alkalmas prototípus fejlesztésre és agilis munkamódszerekhez.
- CRUD (Create, Read, Update, Delete) műveletek implementálása többféle módon, beleértve a vanilla JavaScript, a React komponensek, a Fetch API és az Axios könyvtár használatát.
- Single Page Application (SPA) koncepciók megvalósítása, amelyek lehetővé teszik az oldalváltás nélküli, dinamikus felhasználói élményt.
- Objektumorientált JavaScript (OOJS) programozási paradigma gyakorlati alkalmazása rajzoló alkalmazás formájában.

A projekt különlegessége, hogy nem egyetlen technológiai megoldásra koncentrál, hanem szándékosan többféle megközelítést mutat be párhuzamosan. Ez a megközelítés lehetővé teszi, hogy a felhasználók megértsék az egyes technológiák erősségeit és korlátait, és megalapozott döntést tudjanak hozni arról, hogy melyik eszközt érdemes választaniuk saját projektjeikhez. A magyar feltalálók témaválasztása ráadásul kulturális értéket is képvisel, hiszen felhívja a figyelmet a magyar tudomány és innováció kiemelkedő alakjaira.

> Megjegyzés: A projekt oktatási célokat szolgál, és a valós fejlesztési gyakorlatokat szimulálja. Minden implementáció production-közeli minőséget céloz meg, miközben kellően egyszerű marad ahhoz, hogy a tanulási folyamatot támogassa.

2. Általános architektúra és rendszerterv

A Magyar Feltalálók Webalkalmazás klasszikus kliens-szerver architektúrát követ, amelyben a frontend és a backend komponensek egymástól függetlenül fejleszthetők és telepíthetők. Ez a szétválasztás (separation of concerns) a modern webfejlesztés egyik alapelve, amely biztosítja a rendszer rugalmasságát, karbantarthatóságát és skálázhatóságát. Az architektúra tervezésekor figyelembe vettük a könnyű bővíthetőséget és a komponensek közötti laza csatolás megvalósítását.

2.1 A rendszer két fő komponense

Backend (Python/FastAPI)

A backend réteg feladata az adatok kezelése, az üzleti logika végrehajtása és az API végpontok biztosítása. A szerver oldalon a FastAPI keretrendszert használjuk, amely a Python programozási nyelvhez készült, és kiemelkedő teljesítményt nyújt az aszinkron programozási modell révén. A FastAPI automatikusan generál OpenAPI (Swagger) dokumentációt, ami jelentősen megkönnyíti az API tesztelését és a fejlesztői együttműködést. Az adatbázis-kezelést a MongoDB NoSQL adatbázis-rendszer biztosítja, amelyhez az AsyncIOMotorClient aszinkron driver-en keresztül kapcsolódunk. Ez lehetővé teszi, hogy a szerver egyszerre több kérést is hatékonyan kiszolgáljon anélkül, hogy blokkolná a futási szálat. Az adatok validálását és sorosítását Pydantic modellek végzik, amelyek típusbiztos adatkezelést biztosítanak.

Frontend (JavaScript/React)

A frontend réteg felelős a felhasználói felület megjelenítéséért és az interakciók kezeléséért. A fő keretrendszer a React 18, amely komponens-alapú megközelítést alkalmaz a felhasználói felület építéséhez. A stílusozáshoz a Tailwind CSS utility-first keretrendszert és a shadcn/ui komponenskönyvtárat használjuk, amelyek együtt gyors és konzisztens UI fejlesztést tesznek lehetővé. Az oldalak közötti navigációt a React Router biztosítja, amely kliens oldali routing-ot valósít meg, így az oldalváltások teljes oldalújratöltés nélkül történnek. A build folyamatot a Create React App alapozza meg, amelyet a Craco konfiguráció-kiterjesztéssel egészítettünk ki a testreszabási lehetőségek bővítéséhez. A backend kommunikáció két módon valósul meg: az Axios könyvtárral és a böngésző natív Fetch API-jával.

2.2 Kommunikáció a rétegek között

A frontend és a backend között RESTful API-n keresztül történik a kommunikáció. A frontend HTTP kéréseket küld a backend API végpontjaihoz (GET, POST, PUT, DELETE), a backend pedig JSON formátumban válaszol. Ez a megközelítés platform-független, ami azt jelenti, hogy a backendet bármilyen más frontend technológiával (pl. Angular, Vue.js, mobil alkalmazás) is fel lehetne használni változtatás nélkül. A CORS (Cross-Origin Resource Sharing) middleware biztosítja, hogy a frontend alkalmazás hozzáférhessen a backend erőforrásaihoz, miközben megakadályozza az illetéktelen hozzáférést. A kérések és válaszok JSON formátumot használnak, amely könnyű, ember által is olvasható adatcsere-formátum, és szinte minden modern programozási nyelv natívan támogatja.

2.3 Architekturális döntések és indoklásuk

Az architektúra tervezésekor számos döntést hoztunk, amelyek mindegyike mögött konkrét indokok állnak. A NoSQL adatbázis választása a rugalmas adatstruktúra és a gyors prototípus-fejlesztés igényéből fakadt. A FastAPI keretrendszer mellett a teljesítmény, az automatikus dokumentáció és a Python ökoszisztéma széles körű támogatottsága szólt. A React frontend választását a komponens-alapú architektúra, a hatalmas közösség és a gazdag ökoszisztéma indokolta. A Tailwind CSS mellett a gyors fejlesztési ciklus és az alacsony CSS fájlméret szólt, míg a shadcn/ui az egységes megjelenés biztosítása érdekében került a projektbe.

3. Backend részletes leírása

A backend komponens a rendszer gerincét alkotja, felelős az adatok tárolásáért, a kérések feldolgozásáért és a válaszok előállításáért. A FastAPI keretrendszerre épül, amely a Python nyelvhez készült legmodernebb web keretrendszerek egyike, és különösen jól illeszkedik az aszinkron programozási modellhez. A következőkben részletesen bemutatjuk a backend minden lényeges aspektusát.

3.1 Adatmodell

A rendszer központi entitása az Inventor (Feltaláló) modell, amely a Pydantic könyvtár segítségével került definiálásra. A Pydantic lehetővé teszi a típusellenőrzést futásidőben, automatikus adatvalidálást és JSON sorosítást. Ez azt jelenti, hogy ha egy API kérés hibás adatot tartalmaz (például szöveget egy szám mezőben), a rendszer automatikusan hibaüzenetet küld vissza, még mielőtt az adat eléri az adatbázist.

Az Inventor entitás mezői:

Mező	Típus	Leírás	Kötelező
id	UUID (str)	Egyedi azonosító, automatikusan generált UUID4 formátumban	Igen (auto)
nev	str	A feltaláló teljes neve, magyar ékezetekkel	Igen
szul	int (opcionális)	Születési év, egész szám formátumban (pl. 1800)	Nem
meghal	int (opcionális)	Halálozási év, egész szám formátumban (pl. 1895)	Nem

Az UUID alapú azonosítók használata tudatos döntés volt. Az auto-increment típusú (sorszám alapú) azonosítókkal szemben az UUID-k nem kiszámíthatók, ami biztonsági szempontból előnyös, és elosztott rendszerek esetén is garantáltan egyediek maradnak. A Pydantic modell gondoskodik arról, hogy az opcionális mezők (szul, meghal) hiánya esetén az adat továbbra is érvényes maradjon, így nem minden feltalálóhoz kell teljes életrajzi adatot megadni.

3.2 API végpontok

Az alkalmazás RESTful API végpontokat biztosít, amelyek a HTTP szabvány módszereit (GET, POST, PUT, DELETE) használják az erőforrások kezelésére. Minden végpont JSON formátumban kommunikál, és megfelelő HTTP állapotkódokkal válaszol (200 OK, 201 Created, 404 Not Found, stb.). A végpontok tervezésekor a REST konvenciókat követtük, ami azt jelenti, hogy az URL-ek az erőforrásokat jelölik, a HTTP metódusok pedig a rajtuk végrehajtandó műveleteket.

Végpont	Metódus	Leírás
/api/	GET	Alap információk lekérése az API-ról, verzió és állapot ellenőrzés
/api/inventors	GET	Az összes feltaláló listázása, az adatbázisban tárolt összes rekord visszaadása

Végpont	Metódus	Leírás
/api/inventors/{id}	GET	Egy adott feltaláló részletes adatainak lekérése UUID azonosító alapján
/api/inventors	POST	Új feltaláló létrehozása, a kérés törzsében megadott adatokkal
/api/inventors/{id}	PUT	Meglévő feltaláló adatainak módosítása, a megadott UUID alapján
/api/inventors/{id}	DELETE	Feltaláló törlése az adatbázisból az UUID azonosító alapján
/api/inventors/reset	POST	Adatok visszaállítása az eredeti alapállapotba (seed data)

A reset végpont különösen hasznos oktatási környezetben, mivel lehetővé teszi, hogy a felhasználók kísérletezés után visszaállítsák az adatbázist az eredeti állapotába. Ez biztosítja, hogy a demonstrációk mindig konzisztens kiindulópontról induljanak, és a diákok ne kelljen aggódniuk amiatt, hogy elrontották az adatokat a tanulási folyamat során.

3.3 Adatbázis inicializálás

Az alkalmazás indításakor automatikus adatbázis-inicializálás történik, amelynek során a MongoDB adatbázis feltöltésre kerül 30 neves magyar feltaláló adataival. Ez a seed (vetőmag) adat biztosítja, hogy az alkalmazás azonnal használható legyen az első indítás után, anélkül hogy manuálisan kellene adatokat felvinni. A feltalálók között olyan kiemelkedő személyiségek szerepelnek, mint Jedlik Ányos (a dinamó feltalálója), Bánki Donát (a porlasztó és a vízturbina fejlesztője), Neumann János (a számítógép-architektúra atyja), Rubik Ernő (a Rubik-kocka megalkotója) és Puskás Tivadar (a telefonhírmondó feltalálója).

Az inicializálási folyamat során a rendszer először ellenőrzi, hogy az adatbázis tartalmaz-e már adatokat. Ha az adatbázis üres, automatikusan betölti a seed adatokat. Ha az adatbázis már tartalmaz rekordokat, az inicializálás kihagyásra kerül, így elkerülhető az adatok duplikálása. Ez a megközelítés biztosítja az idempotenciát, vagyis az alkalmazás többszöri újraindítása nem okoz adatinkonzisztenciát.

3.4 Biztonság és konfiguráció

A backend konfiguráció a környezeti változókon (environment variables) keresztül történik, amelyet a python-dotenv könyvtár kezel. Ez a megközelítés biztosítja, hogy az érzékeny adatok (például adatbázis kapcsolódási stringek, API kulcsok) ne kerüljenek bele a forráskódba, és ne legyenek verziókezelőben tárolva. A következő környezeti változók konfigurálhatók:

- MONGO_URL: A MongoDB szerver kapcsolódási string-je, amely tartalmazza a host, port és hitelesítési adatokat. Alapértelmezetten localhost:27017-re mutat fejlesztési környezetben.
- DB_NAME: Az adatbázis neve, amely lehetővé teszi, hogy ugyanazon MongoDB szerveren több független adatbázist tartsunk fenn (pl. fejlesztési, tesztelési és éles adatbázis).
- CORS_ORIGINS: A Cross-Origin Resource Sharing engedélyezett eredeteket definiálja. Ez a beállítás határozza meg, hogy mely domainek férhetnek hozzá az API-hoz, ezzel megelőzve az illetéktelen hozzáférést.

A CORS (Cross-Origin Resource Sharing) middleware konfigurációja biztosítja, hogy a frontend alkalmazás, amely eltérő porton vagy domainen fut, hozzáférhessen a backend API-hoz. Fejlesztési környezetben ez általában a localhost különböző portjait jelenti (pl. a frontend a 3000-es porton, a backend az 8000-es porton fut). Production környezetben a CORS beállításokat szűkíteni kell az éles domain nevekre. A rendszer strukturált naplózást (logging) alkalmaz, amely megkönnyíti a hibakeresést és a rendszer monitorozását.

4. Frontend részletes leírása

A frontend komponens a projekt egyik legösszetettebb része, mivel szándékosan többféle megközelítést mutat be a felhasználói felületek építéséhez. Minden egyes oldal más-más technológiai megoldást alkalmaz ugyanazon alapfeladat (a feltalálók adatainak kezelése) megvalósítására, lehetővé téve a közvetlen összehasonlítást a megközelítések között. A következőkben részletesen bemutatjuk az egyes oldalakat és funkcionalitásukat.

4.1 Főoldal (Index)

A főoldal szolgál a teljes alkalmazás navigációs központjaként. Innen érhetők el az összes alfunkció és fejezet. Az oldal áttekinthető elrendezésben mutatja be a különböző fejezeteket, mindegyikhez rövid leírást és közvetlen hivatkozást biztosítva. A navigáció során a felhasználó a következő fejezeteket érheti el: JavaScript alapú CRUD műveletek (II.), React komponens alapú CRUD (III.), SPA alkalmazások mint a számológép és az amőba játék (IV.), Fetch API alapú CRUD (V.), Axios és React kombinációjú CRUD (VI.), valamint az objektumorientált JavaScript rajzoló alkalmazás (VII.). Az oldal reszponzív kialakítású, ami azt jelenti, hogy asztali számítógépen, tableten és mobiltelefonon egyaránt jól használható.

4.2 JavaScript CRUD (javascript.html)

Ez az oldal a legegyszerűbb, de egyben az alapvető koncepciókat legtisztábban bemutató implementáció. Tisztán vanilla JavaScript-et használ, külső könyvtárak és keretrendszerek nélkül. Az adatokat helyi tömbben tároljuk, ami azt jelenti, hogy az oldal újratöltésekor az adatok elvesznek. Ez a szándékos korlát jól szemlélteti, miért van szükség szerver oldali adattárolásra valós alkalmazásokban.

A DOM (Document Object Model) manipuláció közvetlen JavaScript hívásokkal történik, beleértve az innerHTML, createElement, appendChild és removeChild metódusokat. Az eseménykezelés (event handling) addEventListener-ekkel valósul meg, amelyek a felhasználói interakciókra (kattintás, űrlap beküldés) reagálnak. Az adatok megjelenítése táblázatos formában történik, ahol minden sor tartalmaz szerkesztés és törlés gombokat. Az új feltaláló hozzáadása űrlapon keresztül valósul meg, amely validálja a bevitt adatokat a beküldés előtt. Ez a megközelítés jól demonstrálja, hogy milyen sok kódot igényel egy viszonylag egyszerű funkció megvalósítása puszta JavaScript-tel, és miért jelentenek nagy előrelépést a modern keretrendszerek.

4.3 React CRUD (react.html)

A React alapú CRUD implementáció bemutatja a komponens-alapú fejlesztés előnyeit. A React 18-as verziójának useState hook-ját használjuk az állapotkezeléshez, amely lehetővé teszi, hogy a komponensek állapotváltozásait deklaratív módon kezeljük. A JSX szintaxis segítségével a HTML-szerű kódot közvetlenül a JavaScript-ben írhatjuk, ami intuitívabbá teszi a fejlesztést.

A komponensek hierarchiája gondosan megtervezett: van egy szülő komponens, amely az adatlistát tartja számon, és gyermek komponensek, amelyek az egyes feltalálók adatait jelenítik meg, illetve az űrlapot kezelik. A komponensek közötti kommunikáció props-okon keresztül történik, ahol a szülő komponens callback függvényeket ad át a gyermekeknek. Ez a minta (lifting state up) a React egyik alapvető tervezési mintája, amely biztosítja az egyirányú

adatáramlást a komponensfa mentén.

4.4 SPA oldal (spa.html)

A Single Page Application oldal két interaktív alkalmazást tartalmaz, amelyek bemutatják az SPA koncepció működését. Az SPA lényege, hogy az oldal nem töltődik újra a navigáció során, az összes tartalom dinamikusan cserélődik JavaScript segítségével. Ez gyorsabb és gördülékenyebb felhasználói élményt eredményez a hagyományos többoldalas alkalmazásokhoz képest.

Számológép alkalmazás

A számológép implementáció a négy alapműveletet (összeadás, kivonás, szorzás, osztás) valósítja meg felhasználóbarát felületen. A számológép állapotát JavaScript változók tárolják, a megjelenítés dinamikusan frissül minden gombnyomásra. Az alkalmazás kezeli a speciális eseteket is, mint például a nullával való osztás, a tizedes számok bevitele, és az egymás utáni műveletek láncolása. A felület elrendezése a hagyományos számológépeket idézi, rács (grid) elrendezésben elhelyezett gombokkal.

Tic-Tac-Toe (Amőba) játék

A Tic-Tac-Toe játék két játékos számára készült, akik felváltva helyezik el jeleiket (X és O) egy 3×3-as táblán. A játéklogika JavaScript-ben implementált, és automatikusan felismeri a nyerő pozíciókat (sor, oszlop vagy átló mentén három azonos jel), valamint a döntetlen helyzetet. A játék állapotát egy 9 elemű tömb tárolja, és minden lépés után a rendszer ellenőrzi, hogy van-e győztes. A felület vizuálisan jelzi az aktuális játékos jelét, a nyerő kombinációt, és lehetőséget biztosít a játék újraindítására. Ez a megvalósítás kiválóan demonstrálja az állapotgép (state machine) koncepciót és az eseményvezérelt programozás alkalmazását játékfejlesztésben.

4.5 Fetch API CRUD (fetchapi.html)

Ez az implementáció a böngészőbe beépített Fetch API-t használja a backend szerver kommunikációjára. A Fetch API a modern JavaScript szabvány része, és Promise-alapú aszinkron HTTP kéréseket tesz lehetővé. Az async/await szintaxis használatával a kód olvashatóbb és könnyebben karbantartható, mint a korábbi XMLHttpRequest alapú megoldások.

Az oldalon minden CRUD művelet valódi HTTP kérésen keresztül kommunikál a backend szerverrel. A GET kérések lekérdezik a feltalálók listáját, a POST kérések új feltalálót hoznak létre, a PUT kérések módosítják a meglévő adatokat, a DELETE kérések pedig törlik a kiválasztott rekordot. A válaszok JSON formátumban érkeznek, amelyeket a JavaScript natív JSON.parse() metódusával dolgozunk fel. A hibakezelés try-catch blokkokkal valósul meg, amelyek a hálózati hibákat, a szerver oldali hibákat (4xx, 5xx válaszkódok) és a JSON feldolgozási hibákat egyaránt kezelik. A felhasználó számára érthető hibaüzenetek jelennek meg, amelyek segítenek azonosítani a problémát.

4.6 Axios CRUD (axios.html)

Az Axios könyvtár használatán alapuló implementáció a professzionális szintű HTTP kommunikációt mutatja be React komponensekkel kombinálva. Az Axios a Fetch API-hoz képest számos előnyt kínál: automatikus JSON konverzió (nem kell manuálisan hívni a response.json() metódust), konfigurálható alapértelmezett beállítások (base URL, timeout, fejléc mezők), és

interceptor-ok (elfogók) használata a kérések és válaszok egységes kezelésére.

Az interceptor-ok különösen értékesek az alkalmazásban, mivel lehetővé teszik, hogy minden kimenő kéréshez automatikusan hozzáadódjon a szükséges authentikációs token vagy fejléc, és minden beérkező válasz egységesen kerüljön feldolgozásra. Ha a szerver hiba történik, az interceptor automatikusan naplózza a hibát és egységes hibaformátumot biztosít a komponensek számára. A React komponensekkel történő integrációnál a useState és useEffect hook-okat használjuk: a useEffect biztosítja, hogy az adatok betöltése a komponens csatolásakor (mount) automatikusan megtörténjen, míg a useState tárolja a betöltött adatokat, a betöltés állapotát (loading state) és az esetleges hibákat.

4.7 OOJS rajzoló alkalmazás (oojs.html)

Az objektumorientált JavaScript rajzoló alkalmazás egy teljesen más megközelítést mutat be. Míg az előző oldalak adatkezelési (CRUD) feladatokat valósítottak meg, addig ez az oldal egy interaktív grafikus alkalmazást tartalmaz, amely a HTML5 Canvas elemre épül. Az alkalmazás célja az objektumorientált programozás (OOP) alapelveinek szemléltetése JavaScript-ben.

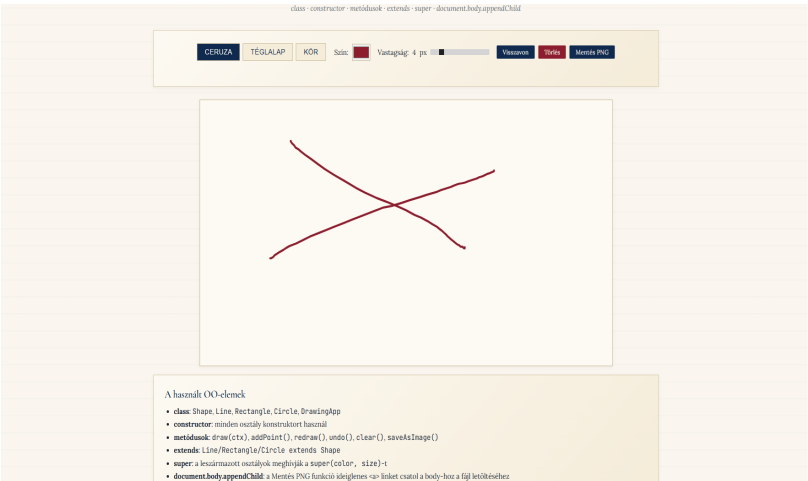
A kód a class, constructor, extends és super kulcsszavakat használja az ES6+ osztályok definiálásához. Az öröklés segítségével különböző alakzat-típusok (kör, négyzet, vonal, szabadkézi rajz) közös ősosztályból származnak, amely az alapvető tulajdonságokat (pozíció, szín, méret) és metódusokat (rajzolás, mozgatás, átméretezés) definiálja. A Canvas API-n keresztül a rajzolás pixelszintű kontrollt biztosít, és a felhasználó egér-eseményekkel (mousedown, mousemove, mouseup) interaktálhat a vászonnal. Az alkalmazás támogatja a szín kiválasztást, az ecsetméret módosítását és a vászon törlését. Ez a megvalósítás demonstrálja, hogy az OOP elvek (kapszulázás, öröklés, polimorfizmus) hogyan alkalmazhatók hatékonyan egy valós JavaScript alkalmazásban.

4.8 Technikai megoldások

Routing

A React Router DOM könyvtár biztosítja a kliens oldali navigációt. A hagyományos szerver oldali routing-gal ellentétben itt az oldalváltás nem igényel szerver kérést, a böngésző URL sávjában megjelenő cím a JavaScript által kerül frissítésre. A BrowserRouter komponens kezeli a history API-t, a Route komponensek definiálják az URL-minta és komponens megfeleltetést, a Link komponensek pedig a navigációs hivatkozásokat biztosítják. A navigációs guard-ok védelmet nyújtanak az illetéktelen hozzáférés ellen, és biztosítják, hogy a felhasználó csak az engedélyezett oldalakra navigálhasson.

OOJS Rajzoló Alk.:



UI komponensek

A felhasználói felület komponensei a shadcn/ui könyvtárra épülnek, amely előre elkészített, testreszabható React komponenseket biztosít. A shadcn/ui különlegessége, hogy a komponensek kódja közvetlenül a projekt forráskódjába kerül (nem külső függőségként), ami teljes kontrollt biztosít a testreszabás felett. A stílusozás Tailwind CSS utility-first megközelítéssel történik, ahol az egyes CSS tulajdonságok osztálynevekként jelennek meg az elemeken (pl. text-blue-500, p-4, flex, rounded-lg). Ez a megközelítés gyorsabb fejlesztést tesz lehetővé, és biztosítja, hogy csak a ténylegesen használt CSS kerüljön be a végleges build-be. Az alkalmazás akadémiai stílusú megjelenést alkalmaz, amely a professzionális dokumentumok és oktatási anyagok vizuális nyelvezetét követi. A reszponzív design biztosítja, hogy az alkalmazás különböző képernyőméreteken is megfelelően jelenjen meg, a Tailwind CSS beépített breakpoint rendszerének segítségével.

Állapotkezelés

Az alkalmazásban az állapotkezelés React useState hook-kal valósul meg komponens szinten. Minden komponens saját belső állapottal rendelkezik, amelyet a useState hook deklarál és a setter függvény módosít. Az adatok komponensek közötti megosztása props drilling mintával történik, ahol a szülő komponens állapotváltozókat és callback függvényeket ad át gyermekeinek a props-okon keresztül. Bár nagyobb alkalmazásoknál ez a minta nehézkessé válhat (és ilyenkor érdemesebb globális állapotkezelő megoldásokat, mint a Context API vagy Redux használni), a projekt méretéhez a komponens szintű állapotkezelés tökéletesen megfelel, és kiválóan demonstrálja az adatáramlás alapelveit React-ben.

Build és fejlesztési eszközök

A projekt a Create React App (CRA) alapra épül, amelyet a Craco (Create React App Configuration Override) konfigurációs eszközzel bővítettünk. A CRA egy jól bevált projekt-sablont biztosít React alkalmazásokhoz, amely tartalmazza a Webpack bundler konfigurációt, a fejlesztői szerver beállításokat és a production build folyamatot. A Craco lehetővé teszi, hogy a CRA belső konfigurációját (Webpack, Babel, ESLint) módosítsuk anélkül, hogy az eject parancsot kellene használnunk, amely visszafordíthatatlanul feltárja az összes konfigurációs fájlt. A Craco-n keresztül health check plugin-okat és development server middleware-eket is beállítottunk, amelyek az alkalmazás állapotát monitorozzák fejlesztés közben.

5. Technikai stack részletesen

A projekt technológiai választásai gondos mérlegelés eredményei. Minden egyes eszköz és könyvtár kiválasztásakor figyelembe vettük a tanulhatóságot, a közösségi támogatottságot, a dokumentáció minőségét és az iparági elterjedtséget. A következőkben részletesen bemutatjuk az összes használt technológiát és az alkalmazásuk indokait.

5.1 Backend technológiák

- FastAPI: Modern, nagy teljesítményű Python web keretrendszer, amely az aszinkron programozási modellre épül. A FastAPI a Starlette web keretrendszerre és a Pydantic adatvalidációs könyvtárra épít, és automatikus OpenAPI dokumentációt generál. A típus-annotációk használata révén a fejlesztői élmény kiemelkedő, és a kód önmagát dokumentálja.
- MongoDB: NoSQL dokumentum-orientált adatbázis, amely JSON-szerű BSON dokumentumokban tárolja az adatokat. A séma nélküli (schema-less) természete rugalmasságot biztosít az adatstruktúra módosításakor, és kiválóan alkalmas prototípus fejlesztésre és agilis módszertanhoz.
- Motor: Aszinkron Python driver a MongoDB-hez, amely az AsyncIO keretrendszerre épül. A Motor lehetővé teszi, hogy a FastAPI aszinkron végpontjai nem blokkoló módon kommunikáljanak az adatbázissal, ami jelentősen javítja a szerver áteresztőképességét.
- Pydantic: Adatvalidációs és sorosítási könyvtár, amely Python típus-annotációkat használ a modellek definiálásához. Automatikusan ellenőrzi a bejövő adatokat, és informatív hibaüzeneteket generál érvénytelen adatok esetén.
- Uvicorn: Villámgyors ASGI (Asynchronous Server Gateway Interface) szerver, amely a FastAPI alkalmazás futtatásáért felelős. Támogatja a HTTP/1.1 és WebSocket protokollokat.
- python-dotenv: Környezeti változók kezelésére szolgáló könyvtár, amely .env fájlokból olvassa be a konfigurációs értékeket, megkönnyítve a fejlesztési és éles környezetek közötti váltást.

5.2 Frontend technológiák

- React 18: A Meta (korábban Facebook) által fejlesztett komponens-alapú UI könyvtár. A 18-as verzió új funkciókat hozott, mint a Concurrent Features, az automatikus batching és a Suspense komponens, amelyek javítják az alkalmazás reakcióképességét.
- React Router: A React ökoszisztéma szabványos routing megoldása, amely kliens oldali navigációt biztosít. A deklaratív routing megközelítés lehetővé teszi, hogy az URL-eket React komponensekre képezzük le.
- Tailwind CSS: Utility-first CSS keretrendszer, amely alacsony szintű CSS osztályokat biztosít közvetlenül a HTML elemeken. A JIT (Just-In-Time) compiler csak a ténylegesen használt osztályokat tartalmazza a végleges CSS fájlban, ami minimális fájlméretet eredményez.
- shadcn/ui: Újrafelhasználható UI komponenskönyvtár, amely a Radix UI primitívekre és a Tailwind CSS-re épül. A komponensek másolás-beillesztéssel kerülnek a projektbe, ami teljes testreszabási lehetőséget biztosít.

- Axios: Promise-alapú HTTP kliens könyvtár, amely a böngészőben és Node.js környezetben egyaránt használható. Beépített támogatást nyújt a kérés/válasz interceptor-okhoz, a kérés visszavonáshoz és az automatikus JSON konverzióhoz.
- Craco: Create React App konfigurációs eszköz, amely lehetővé teszi a Webpack, Babel és ESLint beállítások módosítását az eject parancs használata nélkül.

6. Működési elv és adatfolyam

A Magyar Feltalálók Webalkalmazás működése több lépésből álló adatfolyamra épül, amelyben a frontend és a backend komponensek szorosan együttműködnek. A következőkben részletesen bemutatjuk az adatfolyam minden lépését és az alkalmazott architekturális mintákat.

6.1 Adatfolyam lépésről lépésre

1. Backend inicializálás: Az alkalmazás indításakor a FastAPI szerver betöltődik, létrehozza a MongoDB kapcsolatot az AsyncIOMotorClient segítségével, és ellenőrzi az adatbázis állapotát. Ha az adatbázis üres, automatikusan végrehajtja a seed adatok feltöltését, amelynek során 30 magyar feltaláló adata kerül az adatbázisba. A szerver naplózza az inicializálási folyamat minden lépését.

2. API kérések fogadása: A frontend komponensek HTTP kéréseket küldenek az API végpontokhoz. A kérések tartalmazhatnak URL paramétereket (GET kéréseknél az azonosító), query paramétereket (szűrési feltételek) és body tartalmat (POST és PUT kéréseknél a feltaláló adatai JSON formátumban). A CORS middleware ellenőrzi, hogy a kérés engedélyezett forrásból érkezett-e.

3. Adatfeldolgozás: A backend fogadja a kérést, a Pydantic modell validálja a bejövő adatokat (ellenőrzi a típusokat, a kötelező mezőket és az értéktartományokat), majd végrehajtja a megfelelő adatbázis-műveletet. A Motor aszinkron driver biztosítja, hogy az adatbázis-műveletek ne blokkolják a szerver futási szálát, így más kérések párhuzamosan kiszolgálhatók.

4. Válasz előállítása: A művelet eredménye JSON formátumban kerül visszaküldésre a megfelelő HTTP állapotkóddal. Sikeres művelet esetén a 200 (OK) vagy 201 (Created) kódot használjuk, nem talált erőforrás esetén 404-et (Not Found), validációs hiba esetén 422-t (Unprocessable Entity).

5. Frontend frissítés: A frontend komponensek fogadják a választ, feldolgozzák a JSON adatokat, és frissítik a felhasználói felületet. React esetén az állapot (state) frissítése automatikusan kiváltja a komponens újrarenderelését, biztosítva, hogy a felhasználó mindig az aktuális adatokat lássa.

6.2 Architekturális minták

A projekt több bevált architekturális mintát alkalmaz, amelyek biztosítják a kód szervezettségét és karbantarthatóságát. A RESTful API minta biztosítja, hogy az API végpontok az erőforrások körül szerveződjenek, a HTTP metódusok jelöljék a műveletet, és az állapotkódok egyértelműen közöljék a művelet eredményét. Az MVC (Model-View-Controller) minta adaptált formában jelenik meg: a Pydantic modellek alkotják a Model réteget, a React komponensek a View réteget, a FastAPI route-ok pedig a Controller réteget. A Component Composition minta a React oldalon biztosítja, hogy a kisebb, újrafelhasználható komponensek összeállításával komplexebb felületeket építhessünk. A Separation of Concerns elv szerint a frontend és a backend teljesen független egységekként működnek, kizárólag az API-n keresztül kommunikálnak, ami lehetővé teszi a független fejlesztést és telepítést.

7. Telepítés és futtatás

7.1 Előfeltételek

Az alkalmazás futtatásához az alábbi szoftverek telepítése szükséges a fejlesztői gépen. Az egyes szoftverek verziószámai minimális követelményeket jelölnek, újabb verziók is használhatók, amennyiben visszafelé kompatibilisek.

- Python 3.8 vagy újabb: A backend szerver futtatásához szükséges. A 3.8-as verzió minimális követelmény az async/await szintaxis és az f-string formázás teljes támogatása miatt. Javasolt a legfrissebb stabil verzió (3.11+) használata a jobb teljesítmény és a kibővített típus-annotációk támogatása érdekében.
- Node.js 16 vagy újabb: A frontend fejlesztői szerver és a build eszközök futtatásához szükséges. A Node.js tartalmazza az npm csomagkezelőt, amely a frontend függőségek telepítését végzi. A 18-as vagy 20-as LTS (Long Term Support) verzió használata javasolt.
- MongoDB: A NoSQL adatbázis szerver, amely az adatok tárolásáért felelős. Telepíthető lokálisan (Community Edition), vagy használható felhő alapú szolgáltatásként (MongoDB Atlas). A MongoDB Compass grafikus felületet biztosít az adatok böngészéséhez és kezeléséhez.
- npm vagy yarn: JavaScript csomagkezelő a frontend függőségek kezeléséhez. Az npm a Node.js-sel együtt települ, a yarn alternatív csomagkezelőként használható.

7.2 Backend indítása

A backend szerver indítása előtt létre kell hozni egy Python virtuális környezetet, amely izolálja a projekt függőségeit a rendszer Python telepítésétől. A virtuális környezet létrehozása a 'python -m venv venv' paranccsal történik, aktiválása pedig a 'source venv/bin/activate' (Linux/Mac) vagy 'venv\Scripts\activate' (Windows) paranccsal. A függőségek telepítése a 'pip install -r requirements.txt' paranccsal végezhető el. A szerver indítása a 'uvicorn main:app --reload' paranccsal történik, ahol a --reload kapcsoló biztosítja, hogy a szerver automatikusan újraindul a kód módosításakor. A szerver alapértelmezetten a 8000-es porton indul el, és a http://localhost:8000/docs címen elérhető az automatikusan generált API dokumentáció.

7.3 Frontend indítása

A frontend alkalmazás indításához először telepíteni kell a JavaScript függőségeket az 'npm install' paranccsal, amely a package.json fájlban felsorolt összes csomagot letölti és a node_modules mappába helyezi. A fejlesztői szerver indítása az 'npm start' paranccsal történik, amely a React alkalmazást a 3000-es porton teszi elérhetővé. A fejlesztői szerver automatikus újratöltést (hot reloading) biztosít, ami azt jelenti, hogy a kód módosítása után az oldal automatikusan frissül a böngészőben anélkül, hogy manuálisan kellene újratölteni. Production build készítéséhez az 'npm run build' parancsot kell használni, amely optimalizált, minifikált fájlokat generál a build mappába.

7.4 Környezeti változók beállítása

A projekt gyökérkönyvtárában létre kell hozni egy .env fájlt, amely tartalmazza a szükséges konfigurációs értékeket. Ez a fájl soha nem kerül verziókezelő alá (a .gitignore fájlban szerepel), így az érzékeny adatok védve maradnak. A szükséges változók: MONGO_URL (az adatbázis kapcsolódási címe), DB_NAME (az adatbázis neve), CORS_ORIGINS (az engedélyezett frontend címek vesszővel elválasztva), és REACT_APP_BACKEND_URL (a backend szerver címe a frontend teszteléshez). Fejlesztési környezetben tipikus értékek: MONGO_URL=mongodb://localhost:27017, DB_NAME=inventors_db, CORS_ORIGINS=http://localhost:3000, REACT_APP_BACKEND_URL=http://localhost:8000.

8. Tesztelés

A tesztelés a szoftverfejlesztés egyik legfontosabb lépése, amely biztosítja, hogy az alkalmazás a várt módon működjön, és a módosítások ne okozzanak hibákat a meglévő funkciókban. A Magyar Feltalálók Webalkalmazás mind a backend, mind a frontend oldalon tartalmaz tesztelési megoldásokat, amelyek különböző szintű ellenőrzést biztosítanak.

8.1 Backend tesztelés

A backend tesztelés a pytest keretrendszerrel valósul meg, amely a Python ökoszisztéma egyik legelterjedtebb tesztelési eszköze. A pytest egyszerű szintaxist biztosít a tesztesetek írásához, és gazdag plugin-rendszere révén könnyen bővíthető. A tesztek a requests könyvtárat használják az API végpontok meghívására, szimulálva a frontend kéréseket.

A tesztelési stratégia az API végpontok szisztematikus ellenőrzésére épül. Minden CRUD művelethez tartozik legalább egy teszteset, amely ellenőrzi a sikeres végrehajtást (happy path) és a hibakezelést (error cases). A tesztek a következő aspektusokat vizsgálják:

- Az összes feltaláló listázásának (GET /api/inventors) helyes működése, beleértve a válasz formátumát, a visszaadott adatok teljességét és az üres adatbázis kezelését.
- Új feltaláló létrehozásának (POST /api/inventors) validálása: helyes adatokkal a 201-es válaszkód visszaadása, hiányos adatokkal a megfelelő hibaüzenet generálása.
- Egy adott feltaláló lekérdezésének (GET /api/inventors/{id}) tesztelése: létező és nem létező azonosítókkal, az utóbbi esetben a 404-es válaszkód ellenőrzése.
- Feltaláló adatainak módosítása (PUT /api/inventors/{id}): a módosított adatok helyes mentésének és visszaadásának ellenőrzése.
- Feltaláló törlésének (DELETE /api/inventors/{id}) tesztelése: a sikeres törlés utáni újralekérdezés 404-es választ kell adjon.
- Az adatok visszaállításának (POST /api/inventors/reset) tesztelése: a seed adatok helyes visszaállításának ellenőrzése.

8.2 Frontend tesztelés

A frontend tesztelés a React Testing Library segítségével valósul meg, amely a React komponensek tesztelésére specializálódott eszköz. A könyvtár filozófiája, hogy a teszteknek a felhasználó viselkedését kell szimulálniuk, nem a belső implementációs részleteket vizsgálni. Ez azt jelenti, hogy a tesztek gombokra kattintanak, szöveget írnak be mezőkbe, és ellenőrzik, hogy a képernyőn megjelenik-e a várt tartalom.

A komponens tesztelés során minden React komponenshez készül teszteset, amely ellenőrzi a helyes renderelést (a komponens megjelenik-e a DOM-ban), az állapotváltozásokat (a felhasználói interakciók hatására a UI frissül-e), és a props kezelést (a szülőtől kapott adatok helyesen jelennek-e meg). Az E2E (End-to-End) tesztelés lehetősége is biztosított, amellyel a teljes alkalmazás működése tesztelhető a felhasználó szemszögéből, a frontenddel és a backend szimulációval együttműködve. Az E2E tesztek lehetnek Cypress vagy Playwright alapúak, amelyek automatizált böngésző-tesztelést biztosítanak.

9. Biztonság és teljesítmény

9.1 Biztonsági megoldások

Az alkalmazás biztonsági megoldásai több rétegű védelmet biztosítanak a leggyakoribb webes támadási vektorok ellen. Bár az alkalmazás elsősorban oktatási célokat szolgál, a biztonsági megoldások production-közeli szinten kerültek implementálásra, hogy a tanulók megismerjék a helyes gyakorlatokat.

- CORS konfiguráció: A Cross-Origin Resource Sharing beállítások biztosítják, hogy csak az engedélyezett eredetekből (origins) érkező kéréseket fogadja el a szerver. Ez megakadályozza, hogy rosszindulatú weboldalak illetéktelenül hozzáférjenek az API-hoz. A CORS middleware ellenőrzi a kérés Origin fejlécét, és csak az engedélyezett címekről érkező kéréseket engedi tovább.
- Input validáció (Pydantic): A bejövő adatok validálása a Pydantic modelleken keresztül történik, amely biztosítja, hogy csak a megfelelő típusú és formátumú adatok kerülhessenek az adatbázisba. Ez védelmet nyújt az SQL injection (illetve NoSQL injection) és az XSS (Cross-Site Scripting) támadások ellen is, mivel a nem várt adatokat a rendszer automatikusan elutasítja.
- HTTPS használata production környezetben: Az éles környezetben az alkalmazás HTTPS protokollon keresztül kommunikál, amely titkosítja a kliens és a szerver közötti adatforgalmat. Ez megakadályozza a közbeékelődéses (man-in-the-middle) támadásokat és az adatlopást. A HTTPS tanúsítvány kezelése reverse proxy szerveren (pl. Nginx) vagy felhő szolgáltató szinten történik.
- Környezeti változók használata: Az érzékeny konfigurációs adatok (adatbázis jelszavak, API kulcsok, titkos tokenek) környezeti változókban tárolódnak, és soha nem kerülnek a forráskódba vagy a verziókezelő rendszerbe. A .env fájl a .gitignore-ban szerepel, és a fejlesztők számára egy .env.example sablon áll rendelkezésre.

9.2 Teljesítmény-optimalizálás

Az alkalmazás teljesítményének optimalizálása több szinten történik, a szerver oldali aszinkron programozástól a frontend oldali betöltés-optimalizálásig. Ezek az optimalizálások biztosítják, hogy az alkalmazás gyorsan reagáljon a felhasználói interakciókra, és hatékonyan kezelje az erőforrásokat.

- Aszinkron műveletek: A backend az async/await mintát használja az adatbázis-műveletekhez, ami lehetővé teszi, hogy a szerver egy időben több kérést is kezeljen anélkül, hogy blokkolná a futási szálat. Ez különösen fontos I/O-intenzív műveleteknél (adatbázis-lekérdezések, fájlműveletek), ahol a várakozási idő alatt más kérések is kiszolgálhatók.
- Lazy loading: A frontend oldalon a lazy loading technika biztosítja, hogy csak a szükséges komponensek töltődjenek be, csökkentve az alkalmazás kezdeti betöltési idejét. A React.lazy() és Suspense komponensek segítségével a ritkábban használt oldalak csak akkor töltődnek be, amikor a felhasználó ténylegesen navigál hozzájuk.

- Optimalizált build: A production build során a Webpack bundler minifikálja a JavaScript és CSS fájlokat, eltávolítja a nem használt kódot (tree shaking), és optimalizálja az erőforrásokat. Ez jelentős fájlméret-csökkenést eredményez, ami gyorsabb oldalbetöltést biztosít.
- Health check végpontok: A szerver állapotát monitorozó végpontok lehetővé teszik a rendszer folyamatos felügyeletét, és automatikus riasztásokat generálhatnak, ha a szerver nem válaszol vagy hibás állapotba kerül.

10. Fejlesztési megfontolások

10.1 Skálázhatóság

A projekt architektúrája lehetővé teszi a jövőbeli skálázást. A mikroszolgáltatás architektúrára való áttérés lehetősége adott, ahol az egyes funkciók (feltalálók kezelése, felhasználó-kezelés, értesítések) független szervizekként működnének. Az API versioning biztosítja a visszafelé kompatibilitást új verziók bevezetésekor. A MongoDB indexelés és caching rétegek (pl. Redis) bevezetése tovább javíthatja a teljesítményt nagy adatmennyiség esetén.

10.2 Karbantarthatóság

A moduláris kódstruktúra megkönnyíti a karbantartást és a hibakeresést. A TypeScript-re való migráció lehetősége a típusbiztonság erősítését és a fejlesztői élmény javítását szolgálná. Az automatizált tesztelés és a CI/CD pipeline bevezetése biztosítaná a kód minőségét minden módosítás után. A részletes dokumentáció segíti az új fejlesztők beilleszkedését a projektbe.

10.3 Felhasználói élmény

A reszponzív design biztosítja az alkalmazás használhatóságát minden eszközön. A loading state-ek visszajelzést adnak a felhasználónak az aszinkron műveletek során. A hibakezelés felhasználóbarát üzeneteket jelenít meg a technikai hibaüzenetek helyett. Az accessibility (akadálymentesség) szempontok figyelembevétele biztosítja, hogy az alkalmazás képernyőolvasókkal és billentyűzet-navigációval is használható legyen, megfelelve a WCAG irányelveknek.

11. Összefoglalás és értékelés

A Magyar Feltalálók Webalkalmazás egy átfogó demonstrációja a modern webfejlesztésnek, amely sikeresen ötvözi az oktatási célokat a valós fejlesztési gyakorlatokkal. A projekt bemutatja a teljes stack fejlesztés minden rétegét a frontend felhasználói felülettől a backend API-n át az adatbázis-kezelésig.

A projekt különböző JavaScript paradigmákat mutat be (procedurális, objektumorientált, funkcionális és komponens-alapú), lehetővé téve a fejlesztők számára, hogy megértsék az egyes megközelítések előnyeit és hátrányait. Az API tervezés és implementáció a RESTful konvenciók mentén valósult meg, amely az iparági szabványokat követi. Az adatbázis-kezelés a MongoDB NoSQL megoldásával rugalmas és hatékony adattárolást biztosít.

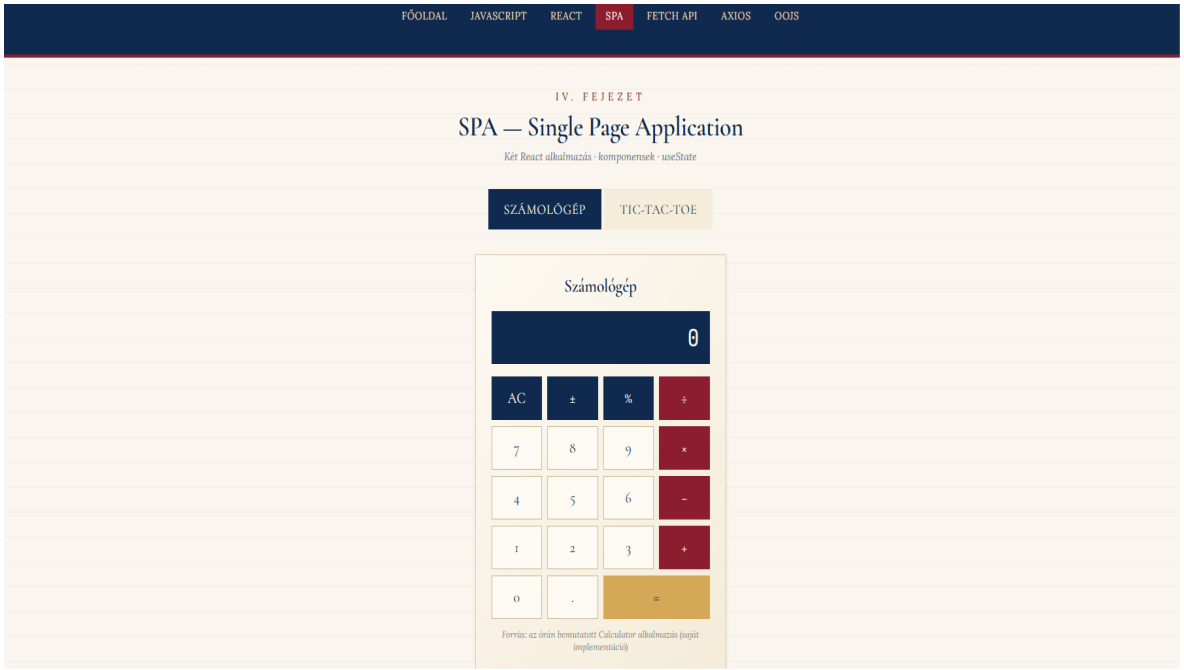
A UI/UX design a modern webalkalmazások vizuális és interakciós elveit követi, a Tailwind CSS és a shadcn/ui könyvtárak felhasználásával. A fejlesztési munkafolyamat a verziókezelés, az automatizált tesztelés és a konfigurálható build rendszer révén a professzionális szoftverfejlesztési gyakorlatokat tükrözi. A tesztelési stratégiák mind a backend, mind a frontend oldalon lefedik a kritikus funkciókat.

> Záró gondolat: Ez a projekt nemcsak technikai tudást közvetít, hanem a szoftverfejlesztési gondolkodásmódot is fejleszti. A különböző megközelítések párhuzamos bemutatása segít kialakítani azt a kritikus szemléletet, amely a megfelelő eszköz kiválasztásához szükséges az adott feladathoz. A magyar feltalálók témája pedig emlékeztet arra, hogy az innováció és a kreativitás a technológiai fejlődés mozgatórugói.

A feladat zárójeles említést tesz a PHP-ról szerveroldali nyelvként, azonban a konkrét feladatpontok (5. és 6.) csak annyit írnak elő, hogy az adatok »szerver oldalon adatbázisban« tárolódjanak. Mivel a fejlesztői környezet Python alapú volt, a szerveroldali réteget FastAPI webkeretrendszerrel és MongoDB dokumentumadatbázissal valósítottam meg. A funkcionalitás megegyezik egy hagyományos PHP+MySQL megoldással: REST API-n keresztül CRUD műveleteket végzünk a kutato adattáblán.

12. Képek

SPA - Számológép



SPA - TicTacToe

Main Page :

Web programozás-1 Előadás Házi feladat

Magyar Feltalálók — Adatbázis kezelő alkalmazás · 2026

FŐOLDAL | JAVASCRIPT | REACT | SPA | FETCH API | AXIOS | OOJS

A · BEADANDÓ · FELADAT

Magyar Feltalálók

— Egy hét fejezetből álló web-alkalmazás —

Ez az alkalmazás bemutatja a Web programozás-1 előadás során elsajátított modern webfejlesztési technológiákat. Az adatbázis a magyar feltalálók adatait tartalmazza (kutató, találmány, kapcsoló táblákkal), és a feladatok ezeken az adatokon végeznek CRUD (*Create*, *Read*, *Update*, *Delete*) műveleteket különböző technológiák segítségével.

FEJEZETEK

A feladat részei

Made with Emergent

II. FEJEZET

JavaScript

TÖMB-ALAPÚ CRUD

Klasszikus JavaScript-tel megvalósított CRUD alkalmazás, melyben az adatok egy tömbben tárolódnak.

Megnyitom →

III. FEJEZET

React

KOMPONENS CRUD

React komponensekkel és useState állapotkezeléssel készült feltaláló-kezelő alkalmazás.

Megnyitom →

IV. FEJEZET

SPA

SZÁMOLÓGÉP & AMŐBA

Egyoldalas alkalmazás (Single Page Application) két React játékkal: Számológép és Tic-Tac-Toe.

Megnyitom →

V. FEJEZET

Fetch API

SZERVEROLDALI TÁROLÁS

JavaScript + Fetch API-val készült CRUD, az adatok MongoDB adatbázisban (FastAPI) tárolódnak.

Megnyitom →

VI. FEJEZET

Axios

REACT + AXIOS

React + Axios alapú alkalmazás, ami a szerver oldali REST API-val kommunikál.

Megnyitom →

VII. FEJEZET

OOJS

OBJEKTUMORIENTÁLT RAJZOLÓ

Objektumorientált JavaScript rajzoló alkalmazás class, constructor, extends, super használatával.

Megnyitom →

FELHASZNÁLT ADATBÁZIS

A választott adattáblák

kutato.txt

Feltalálók: `fkod, nev, szul, meghal`

talalmany.txt

Találmányok: `tkod, talnev`

kapcsol.txt

Kapcsolótábla: `tkod, fkod`

*A CRUD alkalmazások a **kutato.txt** (feltalálók) tábla adatain dolgoznak.*

Készítette: Kovács Kristóf

NEPTUN KÓD: XADNA6

Budapest · MMXXVI · Web Programozás-1

Java Page :

Web programozás-1 Előadás Házi feladat

Magyar Feltalálók — Adatbázis kezelő alkalmazás · 2026

FŐOLDAL | JAVASCRIPT | REACT | SPA | FETCH API | AXIOS | OOJS

II. FEJEZET

JavaScript CRUD

Vanilla JavaScript · Adatok tömbben tárolva · DOM manipuláció

Új feltaláló hozzáadása

Név | Születés | Halálozás (opc.) | Hozzáadás

Kód	Név	Születés	Halálozás	Műveletek
31	Jedlik Ányos István	1800	1895	Szerk. Töröl
9	Bláthy Ottó Titusz	1860	1939	Szerk. Töröl
78	Zipernowsky Károly	1853	1942	Szerk. Töröl
14	Déri Miksa	1854	1938	Szerk. Töröl
5	Bánki Donát	1859	1922	Szerk. Töröl
13	Csonka János	1852	1939	Szerk. Töröl
55	Puskás Tivadar	1844	1893	Szerk. Töröl
35	Kandó Kálmán	1869	1931	Szerk. Töröl
19	Ganz Ábrahám	1814	1867	Szerk. Töröl
44	Mechwart András	1834	1907	Szerk. Töröl
51	Petzval József	1807	1891	Szerk. Töröl
60	Segner János András	1704	1777	Szerk. Töröl
46	Mihály Dénes	1894	1953	Szerk. Töröl
73	Tihanyi Kálmán	1897	1947	Szerk. Töröl
79	Bíró László	1899	1985	Szerk. Töröl
20	Gábor Dénes	1900	1979	Szerk. Töröl
3	Bay Zoltán	1900	1992	Szerk. Töröl
72	Teller Ede	1908	2003	Szerk. Töröl
68	Szilárd Leó	1898	1964	Szerk. Töröl
77	Wigner Jenő	1902	1995	Szerk. Töröl

Megjegyzés: az adatok memóriában (tömbben) tárolódnak — frissítéskor visszaáll a kezdő állapot.

Készítette: Kovács Kristóf - Buday Benedek
NEPTUN KÓD: XADNA6 - OXQLH4

Fetch API :

Web programozás-1 Előadás Házi feladat

Magyar Feltalálók — Adatbázis kezelő alkalmazás · 2026

FŐOLDAL | JAVASCRIPT | REACT | SPA | FETCH API | AXIOS | OOJS

V. FEJEZET

Fetch API CRUD

Vanilla JS + Fetch API · MongoDB szerveroldali tárolás (FastAPI helyettesíti a PHP-t)

Új feltaláló hozzáadása (POST)

Név | Születés | Halálozás (opc.) | Hozzáadás

ID	Név	Születés	Halálozás	Műveletek
a562275d…	Zipernowsky Károly	1853	1942	Szerk. Töröl
8554d4cb…	Déri Miksa	1854	1938	Szerk. Töröl
c2da9a7b…	Bánki Donát	1859	1922	Szerk. Töröl
b1c188f8…	Csonka János	1852	1939	Szerk. Töröl
999a7841…	Puskás Tivadar	1844	1893	Szerk. Töröl
8f785ed6…	Kandó Kálmán	1869	1931	Szerk. Töröl
e8cebd59…	Ganz Ábrahám	1814	1867	Szerk. Töröl
01a9958b…	Mechwart András	1834	1907	Szerk. Töröl
5a32389d…	Petzval József	1807	1891	Szerk. Töröl
086a145d…	Segner János András	1704	1777	Szerk. Töröl
9c31c985…	Mihály Dénes	1894	1953	Szerk. Töröl
b2c571ce…	Tihanyi Kálmán	1897	1947	Szerk. Töröl
6e3d56cc…	Bíró László	1899	1985	Szerk. Töröl
eb841661…	Gábor Dénes	1900	1979	Szerk. Töröl
998fd6d0…	Bay Zoltán	1900	1992	Szerk. Töröl
50f74c96…	Teller Ede	1908	2003	Szerk. Töröl
1b41a1d6…	Szilárd Leó	1898	1964	Szerk. Töröl
e7c40a6f…	Wigner Jenő	1902	1995	Szerk. Töröl
aee2383f…	Neumann János	1903	1957	Szerk. Töröl
193e434d…	Békésy György	1899	1972	Szerk. Töröl
4d5c4f41…	Szent-Györgyi Albert	1893	1986	Szerk. Töröl
dd5f39ca…	Kármán Tódor	1881	1963	Szerk. Töröl
ae2e94e5…	Goldmark Péter Károly	1906	1977	Szerk. Töröl
a13d5c38…	Kemény János György	1926	1992	Szerk. Töröl
e0a3541f…	Bródy Imre	1891	1944	Szerk. Töröl
e95685be…	Just Sándor	1874	1937	Szerk. Töröl
917cfa72…	Frommer Rudolf	1868	1936	Szerk. Töröl
80b4286e…	Hanaman Ferenc	1878	1941	Szerk. Töröl
e3f88647…	TEST_AxiosAdd	1950	2020	Szerk. Töröl
955e2f42…	TEST_FetchAdd	1950	2020	Szerk. Töröl
1f0ef36e…	Fasz János	1990	2000	Szerk. Töröl

Endpointok: `GET / POST / PUT / DELETE {API}/inventors`

Készítette: Kovács Kristóf - Buday Benedek
NEPTUN KÓD: XADNA6 - OXQLH4